```
from google.colab import drive
drive.mount('/content/drive')


```

```
Mounted at /content/drive
```

```
# 2. Add the directory containing 'models.py' to Python's path
import sys
sys.path.append('/content/drive/MyDrive/') # Assuming 'models.py' is directly in MyDrive

# 3. Import the module
import models
from models import *
```

```
import argparse
import os
import time
import shutil

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torch.backends.cudnn as cudnn


import torchvision
import torchvision.transforms as transforms

from models import *


global best_prec
use_gpu = torch.cuda.is_available()
print('=> Building model...')



batch_size = 128
model_name = "VGG16_quant_8x8"
model = VGG16_quant_8x8()

print(model)

normalize = transforms.Normalize(mean=[0.491, 0.482, 0.447], std=[0.247, 0.243, 0.262])


train_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        normalize,
    ]))
trainloader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=2)


test_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transforms.Compose([
        transforms.ToTensor(),
        normalize,
    ]))

testloader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size, shuffle=False, num_workers=2)


print_freq = 100 # every 100 batches, accuracy printed. Here, each batch includes "batch_size" data points
# CIFAR10 has 50,000 training data, and 10,000 validation data.

def train(trainloader, model, criterion, optimizer, epoch):
    batch_time = AverageMeter()
    data_time = AverageMeter()
```

```
    losses = AverageMeter()
    top1 = AverageMeter()

    model.train()

    end = time.time()
    for i, (input, target) in enumerate(trainloader):
        # measure data loading time
        data_time.update(time.time() - end)

        input, target = input.cuda(), target.cuda()

        # compute output
        output = model(input)
        loss = criterion(output, target)

        # measure accuracy and record loss
        prec = accuracy(output, target)[0]
        losses.update(loss.item(), input.size(0))
        top1.update(prec.item(), input.size(0))

        # compute gradient and do SGD step
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # measure elapsed time
        batch_time.update(time.time() - end)
        end = time.time()


        if i % print_freq == 0:
            print('Epoch: [{0}][{1}/{2}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Data {data_time.val:.3f} ({data_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   epoch, i, len(trainloader), batch_time=batch_time,
                   data_time=data_time, loss=losses, top1=top1))


def validate(val_loader, model, criterion ):
    batch_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    # switch to evaluate mode
    model.eval()

    end = time.time()
    with torch.no_grad():
        for i, (input, target) in enumerate(val_loader):

            input, target = input.cuda(), target.cuda()

            # compute output
            output = model(input)
            loss = criterion(output, target)

            # measure accuracy and record loss
            prec = accuracy(output, target)[0]
            losses.update(loss.item(), input.size(0))
            top1.update(prec.item(), input.size(0))

            # measure elapsed time
            batch_time.update(time.time() - end)
            end = time.time()

            if i % print_freq == 0:  # This line shows how frequently print out the status. e.g., i%5 => every 5 batch
                print('Test: [{0}/{1}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   i, len(val_loader), batch_time=batch_time, loss=losses,
                   top1=top1))

    print(' * Prec {top1.avg:.3f}% '.format(top1=top1))
    return top1.avg


def accuracy(output, target, topk=(1,)):
```

```
    """Computes the precision@k for the specified values of k"""
    maxk = max(topk)
    batch_size = target.size(0)

    _, pred = output.topk(maxk, 1, True, True)
    pred = pred.t()
    correct = pred.eq(target.view(1, -1).expand_as(pred))

    res = []
    for k in topk:
        correct_k = correct[:k].view(-1).float().sum(0)
        res.append(correct_k.mul_(100.0 / batch_size))
    return res


class AverageMeter(object):
    """Computes and stores the average and current value"""
    def __init__(self):
        self.reset()

    def reset(self):
        self.val = 0
        self.avg = 0
        self.sum = 0
        self.count = 0

    def update(self, val, n=1):
        self.val = val
        self.sum += val * n
        self.count += n
        self.avg = self.sum / self.count


def save_checkpoint(state, is_best, fdir):
    filepath = os.path.join(fdir, 'checkpoint.pth')
    torch.save(state, filepath)
    if is_best:
        shutil.copyfile(filepath, os.path.join(fdir, 'model_best.pth.tar'))


def adjust_learning_rate(optimizer, epoch):
    """For resnet, the lr starts from 0.1, and is divided by 10 at 80 and 120 epochs"""
    adjust_list = [100, 150, 180]
    if epoch in adjust_list:
        for param_group in optimizer.param_groups:
            param_group['lr'] = param_group['lr'] * 0.1

#model = nn.DataParallel(model).cuda()
#all_params = checkpoint['state_dict']
#model.load_state_dict(all_params, strict=False)
#criterion = nn.CrossEntropyLoss().cuda()
#validate(testloader, model, criterion)
```

```
    (31): ReLU(inplace=True)
    (32): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (33): QuantConv2d(
      256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (34): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (35): ReLU(inplace=True)
    (36): QuantConv2d(
      512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (37): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (38): ReLU(inplace=True)
    (39): QuantConv2d(
      512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (40): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (41): ReLU(inplace=True)
    (42): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (43): AvgPool2d(kernel_size=1, stride=1, padding=0)
  )
  (classifier): Linear(in_features=512, out_features=10, bias=True)
)
100%|██████████| 170M/170M [00:02<00:00, 58.1MB/s]
```

```
1
2 model.features[0] ## Modified layer
```

```
QuantConv2d(
  3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
```

```
1 # # This cell is from the website
2
3 # lr = 1e-2
4
5
6
7 # weight_decay = 1e-4
8 # epochs = 200
9 # best_prec = 0
10
11 # model = model.cuda()
12 # criterion = nn.CrossEntropyLoss().cuda()
13 # optimizer = torch.optim.SGD(model.parameters(), lr=lr, momentum = 0.94, weight_decay=weight_decay) #changed momentum a
14 # # weight decay: for regularization to prevent overfitting
15
```

```
1
2
3 # if not os.path.exists('result'):
4 #     os.makedirs('result')
5
6 # fdir = 'result/'+str(model_name)
7
8 # if not os.path.exists(fdir):
9 #     os.makedirs(fdir)
10
11
```

```
1
2 # for epoch in range(0, epochs):
3 #     adjust_learning_rate(optimizer, epoch)
4
5 #     train(trainloader, model, criterion, optimizer, epoch)
6
7 #     # evaluate on test set
8 #     print("Validation starts")
9 #     prec = validate(testloader, model, criterion)
10
11 #     # remember best precision and save checkpoint
12 #     is_best = prec > best_prec
13 #     best_prec = max(prec,best_prec)
14 #     print('best acc: {:1f}'.format(best_prec))
15 #     if is_best:
16 #       save_checkpoint({
17 #         'epoch': epoch + 1,
18 #         'state_dict': model.state_dict(),
19 #         'best_prec': best_prec,
20 #         'optimizer': optimizer.state_dict(),
```

```
#     }, is_best, fdir)

#     if prec > 90: break

```

```
# !cp -r  /content/result/VGG16_quant_8x8 /content/drive/MyDrive/result

```

```
PATH = '/content/drive/MyDrive/result/VGG16_quant_8x8/model_best.pth.tar'
checkpoint = torch.load(PATH)
model.load_state_dict(checkpoint['state_dict'])
device = torch.device("cuda")

model.cuda()
model.eval()

test_loss = 0
correct = 0

with torch.no_grad():
    for data, target in testloader:
        data, target = data.to(device), target.to(device) # loading to GPU
        output = model(data)
        pred = output.argmax(dim=1, keepdim=True)
        correct += pred.eq(target.view_as(pred)).sum().item()

test_loss /= len(testloader.dataset)

print('\nTest set: Accuracy: {}/{} ({:.0f}%)\n'.format(
        correct, len(testloader.dataset),
        100. * correct / len(testloader.dataset)))
```

```
Test set: Accuracy: 1700/10000 (17%)
```

```
class SaveOutput:
    def __init__(self):
        self.outputs = []
    def __call__(self, module, module_in):
        self.outputs.append(module_in)
    def clear(self):
        self.outputs = []

######### Save inputs from selected layer ##########
save_output = SaveOutput()
i = 0
count = 0
for layer in model.modules():
    i = i+1
    if isinstance(layer, QuantConv2d):
        count += 1
        print(i,"-th layer prehooked \n", layer, count) #'count' here is just for me to check the index of save_output.
        layer.register_forward_pre_hook(save_output)
#####################################################

dataiter = iter(testloader)
images, labels = next(dataiter)
images = images.to(device)
images_abs = torch.abs(images) # taking absolute value of image input since our feature map activations are assumed to
out = model(images_abs)
```

```
3 -th layer prehooked
 QuantConv2d(
  3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 1
7 -th layer prehooked
 QuantConv2d(
  64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 2
12 -th layer prehooked
 QuantConv2d(
  64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 3
16 -th layer prehooked
 QuantConv2d(
  128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 4
```

```
21 -th layer prehooked
 QuantConv2d(
  128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 5
25 -th layer prehooked
 QuantConv2d(
  256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 6
29 -th layer prehooked
 QuantConv2d(
  256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 7
34 -th layer prehooked
 QuantConv2d(
  256, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 8
38 -th layer prehooked
 QuantConv2d(
  8, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 9
41 -th layer prehooked
 QuantConv2d(
  8, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 10
46 -th layer prehooked
 QuantConv2d(
  256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 11
50 -th layer prehooked
 QuantConv2d(
```

```
print(images.shape)
```

```
torch.Size([128, 3, 32, 32])
```

```
weight_q = model.features[0].weight_q
w_alpha = model.features[0].weight_quant.wgt_alpha
w_bit = 4

weight_int = weight_q / (w_alpha / (2**(w_bit-1)-1))
#print(weight_int)
```

```
act = images_abs
act_alpha  = model.features[0].act_alpha
act_bit = 4
act_quant_fn = act_quantization(act_bit)

act_q = act_quant_fn(act, act_alpha)

act_int = act_q / (act_alpha / (2**act_bit-1))
print(torch.all(act_int >= 0)) ## all are positive
```

```
tensor(True, device='cuda:0')
```

```
conv_int = torch.nn.Conv2d(in_channels = 3, out_channels=64, kernel_size = 3, padding=1)
conv_int.weight = torch.nn.parameter.Parameter(weight_int)
relu = torch.nn.ReLU()
conv_int.bias = model.features[0].bias
output_int = conv_int(act_int) # The most important thing
output_recovered = output_int * (act_alpha / (2**act_bit-1)) * (w_alpha / (2**(w_bit-1)-1))
output_recovered_relu = relu(output_recovered)
#print(output_recovered)
```

```
print(save_output.outputs[1][0].shape)
difference = abs(save_output.outputs[1][0] - output_recovered_relu )
print(difference.mean())  ## It should be small, e.g.,2.3 in my trainned model
```

```
torch.Size([128, 64, 32, 32])
tensor(2.7382, device='cuda:0', grad_fn=<MeanBackward0>)
```

```
import torch
import torch.nn.functional as F

# ----------------------------------------------------
# 1. Conv hyperparameters – EDIT THESE TO MATCH YOUR LAYER
# ----------------------------------------------------
```

```
stride = 1        # or (1, 1)
padding = 1       # use 0 if you had no padding
dilation = 1
groups = 1

use_bias = False  # set True and define bias_int if your conv has bias
bias_int = None   # e.g. torch.randint(..., size=(64,), dtype=torch.int16)

# If you do integer scaling (e.g., >> shift_bits after accumulation)
apply_shift = False
shift_bits = 3    # example: right shift by 3

# ------------------------------------------------
# 2. Basic sanity prints
# ------------------------------------------------
print("act_int.shape    :", act_int.shape)     # [128, 3, 32, 32]
print("weight_int.shape :", weight_int.shape)  # [64, 3, 3, 3]
print("output_int.shape :", output_int.shape)  # [128, 64, 32, 32]

batch = 5
out_c = 1
h = 0
w = 0

# ------------------------------------------------
# 3. Float conv2d reference using act_int & weight_int
# ------------------------------------------------
act_f = act_int.to(torch.float32)
weight_f = weight_int.to(torch.float32)

bias_f = None
if use_bias and bias_int is not None:
    bias_f = bias_int.to(torch.float32)

out_f = F.conv2d(
    act_f, weight_f,
    bias=bias_f,
    stride=stride,
    padding=padding,
    dilation=dilation,
    groups=groups,
)

print("F.conv2d reference (float):", out_f[batch, out_c, h, w].item())
print("Stored output_int          :", output_int[batch, out_c, h, w].item())

# ------------------------------------------------
# 4. Manual integer convolution using act_int & weight_int
# ------------------------------------------------
# We need to mimic the same padding as conv2d.
# F.pad pads in the order (left, right, top, bottom).
if isinstance(padding, int):
    pad_left = pad_right = pad_top = pad_bottom = padding
else:
    # if you used tuple padding, adjust this section
    pad_left = pad_right = padding[1] if isinstance(padding, tuple) else padding
    pad_top  = pad_bottom = padding[0] if isinstance(padding, tuple) else padding

act_padded = F.pad(act_int, (pad_left, pad_right, pad_top, pad_bottom))

# Kernel size
_, in_channels, kH, kW = weight_int.shape

# For stride=1, dilation=1, the input top-left index for output (h, w) is:
h_in = h * stride
w_in = w * stride

# Extract 3x3 (or kH x kW) patch from padded input
patch_int = act_padded[batch, :, h_in:h_in + kH, w_in:w_in + kW]   # [Cin, kH, kW]
kernel_int = weight_int[out_c]                                     # [Cin, kH, kW]

# Integer accumulation in int32 to avoid overflow
acc_int32 = (patch_int.to(torch.int32) * kernel_int.to(torch.int32)).sum()

if use_bias and bias_int is not None:
    acc_int32 = acc_int32 + int(bias_int[out_c].item())

print("Manual integer accumulator :", acc_int32.item())

# ------------------------------------------------
# 5. Optional: apply integer scaling / shifting and clamp to int16
# ------------------------------------------------
```

```
if apply_shift:
    # arithmetic right shift for signed values
    acc_shifted = acc_int32 >> shift_bits
else:
    acc_shifted = acc_int32

# Clamp to int16 range
acc_int16 = torch.clamp(acc_shifted, -32768, 32767).to(torch.int16)

print("Manual quantized int16     :", acc_int16.item())
print("Stored output_int          :", output_int[batch, out_c, h, w].item())
```

```
act_int.shape    : torch.Size([128, 3, 32, 32])
weight_int.shape : torch.Size([64, 3, 3, 3])
output_int.shape : torch.Size([128, 64, 32, 32])
F.conv2d reference (float): 28.0
Stored output_int          : 28.0
Manual integer accumulator : 28
Manual quantized int16     : 28
Stored output_int          : 28.0
```

```
batch = 5
X = act_padded[batch,:,:,:]  # pick only one input C_in, ni, nj
w_int = torch.reshape(weight_int, (weight_int.size(0), weight_int.size(1), -1))  # out_ch = 64, in_ch = 3, kij = 9
W = w_int
out_int = output_int[batch]
out_int = torch.reshape(out_int, (out_int.size(0), -1))

```

```
print(X.shape)
print(W.shape)
```

```
torch.Size([3, 34, 34])
torch.Size([64, 3, 9])
```

```
batch = 5
in_channels = 3
bit_precision = 4
h = 3
k_size = 3
array_size = 8
kij = 9

for i in range(in_channels):
  fname = f"activation_in_ch{i}.txt"
  file = open(fname, 'w') #write to file
  for j in range(h):
    for k in range(h):
      for l in range(array_size):
        #print(round(X[i,j,7-l+k].item()))
        X_bin = '{0:04b}'.format(round(X[i,j,7-l+k].item()))
        for m in range(bit_precision):
              file.write(X_bin[m])
      file.write('\n')

  file.close()

w_check = []

for i in range(in_channels):
  fname = f"weight_in_ch{i}.txt"
  file = open(fname, 'w') #write to file
  for ker in range(kij):
    for j in range(array_size):
      if (7-j == 2):
        w_check.append(W[7-j,i,ker].item())
        print(f"o_channel: {7-j}, in_channel {i}, kernel {ker}, value {w_check[-1]}")
      if (round(W[7-j,i,k].item()) >= 0):
          W_bin = '{0:04b}'.format(round(W[7-j,i,ker].item()))
      else:
          W_bin = '{0:04b}'.format(round(W[7-j,i,ker].item())+16)
      for k in range(bit_precision):
          file.write(W_bin[k])
    file.write('\n')
  file.close()

```

```
o_channel: 2, in_channel 0, kernel 0, value 0.0
o_channel: 2, in_channel 0, kernel 1, value 0.0
o_channel: 2, in_channel 0, kernel 2, value 0.0
o_channel: 2, in_channel 0, kernel 3, value 0.0
o_channel: 2, in_channel 0, kernel 4, value 0.0
```

```
o_channel: 2, in_channel 0, kernel 5, value 0.0
o_channel: 2, in_channel 0, kernel 6, value 0.0
o_channel: 2, in_channel 0, kernel 7, value 0.0
o_channel: 2, in_channel 0, kernel 8, value 0.0
o_channel: 2, in_channel 1, kernel 0, value 0.0
o_channel: 2, in_channel 1, kernel 1, value 0.0
o_channel: 2, in_channel 1, kernel 2, value 0.0
o_channel: 2, in_channel 1, kernel 3, value 0.0
o_channel: 2, in_channel 1, kernel 4, value 0.0
o_channel: 2, in_channel 1, kernel 5, value 0.0
o_channel: 2, in_channel 1, kernel 6, value -0.0
o_channel: 2, in_channel 1, kernel 7, value -0.0
o_channel: 2, in_channel 1, kernel 8, value -0.0
o_channel: 2, in_channel 2, kernel 0, value -0.0
o_channel: 2, in_channel 2, kernel 1, value -0.0
o_channel: 2, in_channel 2, kernel 2, value -0.0
o_channel: 2, in_channel 2, kernel 3, value -7.0
o_channel: 2, in_channel 2, kernel 4, value -7.0
o_channel: 2, in_channel 2, kernel 5, value -7.0
o_channel: 2, in_channel 2, kernel 6, value -7.0
o_channel: 2, in_channel 2, kernel 7, value -7.0
o_channel: 2, in_channel 2, kernel 8, value -7.0
```

```
1 out_int.shape #out_ch, o_nij
```

```
torch.Size([64, 1024])
```

```
1 o_nij = 8
2 o_chan = 8
3 bit_precision = 16
4
5 fname = "out.txt"
6 file = open(fname, 'w') #write to file
7 out_int = relu(out_int) #relu(out_int)
8 for i in range(o_nij):
9   for o in range(o_chan):
10    #out_int[7-o,7-i]
11    if (7-o == 1 and 7-i == 0): print(out_int[7-o,7-i].item())
12    if (out_int[7-o,7-i].item() >= 0):
13        Out_bin = '{0:016b}'.format(round(out_int[7-o,7-i].item()))
14    else:
15        Out_bin = '{0:016b}'.format(round(out_int[7-o,7-i].item())+65536)
16    for k in range(bit_precision):
17        file.write(Out_bin[k])
18  file.write('\n')
19 file.close()
```

```
28.0
```

```
1 print(out_int[1,1])
2 print(out_int.shape)
3 print(X.shape)
4 print(W.shape)
```

```
tensor(56., device='cuda:0', grad_fn=<SelectBackward0>)
torch.Size([64, 1024])
torch.Size([3, 34, 34])
torch.Size([64, 3, 9])
```

```
1 (X[:,0:3,1:4]*W[1,:,:].reshape(3,3,3)).sum() #equivalent  to out_ch = 1 and o_ij = 0
```

```
tensor(56., device='cuda:0', grad_fn=<SumBackward0>)
```

```
1 torch.set_printoptions(profile="full")
2 print(w_check[18:])
3 print(W[2,2,:].reshape(3,3)) #o_ch = 2, in_ch = 2, 0:9
4 weight_int[2,2,:,:]
```

```
[-0.0, -0.0, -0.0, -7.0, -7.0, -7.0, -7.0, -7.0, -7.0]
tensor([[-0., -0., -0.],
        [-7., -7., -7.],
        [-7., -7., -7.]], device='cuda:0', grad_fn=<ViewBackward0>)
tensor([[-0., -0., -0.],
        [-7., -7., -7.],
        [-7., -7., -7.]], device='cuda:0', grad_fn=<SelectBackward0>)
```

```
1 # # Make sure these match your actual conv layer
2 # stride = 1
3 # padding = 0
4 # dilation = 1
5 # groups = 1
6 # bias = None  # or your bias tensor if you have one
```

```
 7
 8 # # Convert to float just for comparison
 9 # act_f = act_int.to(torch.float32)
10 # weight_f = weight_int.to(torch.float32)
11
12 # out_check = F.conv2d(
13 #     act_f, weight_f,
14 #     bias=bias,
15 #     stride=stride,
16 #     padding=padding,
17 #     dilation=dilation,
18 #     groups=groups,
19 # )
20
21 # print("F.conv2d value:", out_check[0, 6, 0, 0].item())
22 # print("Your stored output:", output_int[0, 6, 0, 0].item())
23
24 # print(output_int.shape) #[128, 64, 32, 32]
25 # print(act_int.shape) #[128, 3, 32, 32]
26 # print(weight_int.shape) #[64, 3, 3, 3]
27
28 # batch = 0
29 # out_c = 6
30 # h = 0
31 # w = 0
32
33 # # Extract the relevant input patch (3 input channels, 3x3 window)
34 # # Input shape: [batch, in_channels, H, W]
35 # patch = act_int[batch, :, h:h+3, w:w+3]  # shape [3, 3, 3]
36
37 # # Extract the kernel for output channel 6
38 # kernel = weight_int[out_c]  # shape [3, 3, 3]
39
40 # # Elementwise multiply and sum
41 # manual_value = (patch * kernel).sum()
42
43 # print("Manual conv value:", manual_value)
44 # print("Output map value :", output_int[batch, out_c, h, w])
```

```
 1 |
 2
 3 # act_int.size = torch.Size([128, 64, 32, 32])  <- batch_size, input_ch, ni, nj
 4 a_int = act_int[0,:,:,:]  # pick only one input out of batch
 5 # a_int.size() = [64, 32, 32]
 6
 7 # conv_int.weight.size() = torch.Size([64, 64, 3, 3])  <- output_ch, input_ch, ki, kj
 8 w_int = torch.reshape(weight_int, (weight_int.size(0), weight_int.size(1), -1))  # merge ki, kj index to kij
 9 # w_int.weight.size() = torch.Size([64, 64, 9])
10
```

```
 1 # w_int.shape
```

```
torch.Size([64, 3, 9])
```

```
 1
 2 # padding = 1
 3 # stride = 1
 4 # array_size = 8 # row and column number
 5
 6 # nig = range(a_int.size(1))  ## ni group
 7 # njg = range(a_int.size(2))  ## nj group
 8
 9 # icg = range(int(w_int.size(1)))  ## input channel
10 # ocg = range(int(w_int.size(0)))  ## output channel
11
12 # ic_tileg = range(int(len(icg)/array_size))
13 # oc_tileg = range(int(len(ocg)/array_size))
14
15 # kijg = range(w_int.size(2))
16 # ki_dim = int(math.sqrt(w_int.size(2)))  ## Kernel's 1 dim size
17
18 # ######## Padding before Convolution #######
19 # a_pad = torch.zeros(len(icg), len(nig)+padding*2, len(nig)+padding*2).cuda()
20 # # a_pad.size() = [64, 32+2pad, 32+2pad]
21 # a_pad[ :, padding:padding+len(nig), padding:padding+len(njg)] = a_int.cuda()
22 # a_pad = torch.reshape(a_pad, (a_pad.size(0), -1))
23 # # a_pad.size() = [64, (32+2pad)*(32+2pad)]
24
25
26 # a_tile = torch.zeros(len(ic_tileg), array_size,    a_pad.size(1)).cuda()
```

```
# w_tile = torch.zeros(len(oc_tileg)*len(ic_tileg), array_size, array_size, len(kijg)).cuda()

# for ic_tile in ic_tileg:
#     a_tile[ic_tile,:,:] = a_pad[ic_tile*array_size:(ic_tile+1)*array_size,:]

# for ic_tile in ic_tileg:
#     for oc_tile in oc_tileg:
#         w_tile[oc_tile*len(oc_tileg) + ic_tile,:,:,:] = w_int[oc_tile*array_size:(oc_tile+1)*array_size, ic_tile*arra



# ##########################################

# p_nijg = range(a_pad.size(1)) ## psum nij group

# psum = torch.zeros(len(ic_tileg), len(oc_tileg), array_size, len(p_nijg), len(kijg)).cuda()

# for kij in kijg:
#     for ic_tile in ic_tileg:       # Tiling into array_sizeXarray_size array
#         for oc_tile in oc_tileg:   # Tiling into array_sizeXarray_size array
#             for nij in p_nijg:       # time domain, sequentially given input
#                     m = nn.Linear(array_size, array_size, bias=False)
#                     m.weight = torch.nn.Parameter(w_int[oc_tile*array_size:(oc_tile+1)*array_size, ic_tile*array_size
#                     #m.weight = torch.nn.Parameter(w_tile[len(oc_tileg)*oc_tile+ic_tile,:,:,kij])
#                     psum[ic_tile, oc_tile, :, nij, kij] = m(a_tile[ic_tile,:,nij]).cuda()

```

```
# print(out.shape)
# print(o_ni_dim)
```

```
torch.Size([64, 1024])
32
```

```
# import math

# a_pad_ni_dim = int(math.sqrt(a_pad.size(1))) # 32

# o_ni_dim = int((a_pad_ni_dim - (ki_dim- 1) - 1)/stride + 1)
# o_nijg = range(o_ni_dim**2)

# out = torch.zeros(len(ocg), len(o_nijg)).cuda()


# ### SFP accumulation ###
# for o_nij in o_nijg:
#     for kij in kijg:
#         for ic_tile in ic_tileg:
#             for oc_tile in oc_tileg:
#                 out[oc_tile*array_size:(oc_tile+1)*array_size, o_nij] = out[oc_tile*array_size:(oc_tile+1)*array_size
#                 psum[ic_tile, oc_tile, :, int(o_nij/o_ni_dim)*a_pad_ni_dim + o_nij%o_ni_dim + int(kij/ki_dim)*a_pad_n
#                 ## 4th index = (int(o_nij/30)*32 + o_nij%30) + (int(kij/3)*32 + kij%3)
```

```
# out_2D = torch.reshape(out, (out.size(0), o_ni_dim, -1))
# difference = (out_2D - output_int[:,:,:])
# print(difference.sum())
```

```
tensor(440657., device='cuda:0', grad_fn=<SumBackward0>)
```

```
# out.shape

```

```
torch.Size([8, 16])
```

```
# ####### Store activation ####################
# tile_id = 0
# nij = 0 # just a random number
# X = a_tile[tile_id,:,:]  # [tile_num, array row num, time_steps]

# bit_precision = 4
# file = open('activation_tile0.txt', 'w') #write to file
# file.write('#time0row7[msb-lsb],time0row6[msb-lst],....,time0row0[msb-lst]#\n')
# file.write('#time1row7[msb-lsb],time1row6[msb-lst],....,time1row0[msb-lst]#\n')
# file.write('#................#\n')

# for i in range(X.size(1)):  # time step
#     for j in range(X.size(0)): # row #
#         X_bin = '{0:04b}'.format(round(X[7-j,i].item()))
#         for k in range(bit_precision):
#             #print(X_bin[k])
#             file.write(X_bin[k])
```

```
#         #file.write(' ')  # for visibility with blank between words, you can use
#     file.write('\n')
# file.close() #close file

```

```
# print(a_tile.shape)
# print(X.shape, nij)
```

```
torch.Size([1, 8, 36])
torch.Size([8, 36]) 0
```

```
# ####### store weight(kij) ###############
# tile_id = 0
# kij = 0
# bit_precision = 4


# for kij in range (9):
#     file_name = f'weight_itile0_otile0_kij{kij}.txt'
#     W = w_tile[tile_id,:,:,kij] # w_tile[tile_num, array col num, array row num, kij]

#     file = open(file_name, 'w')  #write to file
#     file.write('#col0row7[msb-lsb],col0row6[msb-lst],....,col0row0[msb-lst]#\n')
#     file.write('#col1row7[msb-lsb],col1row6[msb-lst],....,col1row0[msb-lst]#\n')
#     file.write('#................#\n')

#     for i in range(W.size(1)):  # time step
#         for j in range(W.size(0)): # row #
#             if (W[7-j, i].item() >= 0):
#                 W_bin = '{0:04b}'.format(round(W[7-j,i].item()))
#             else:
#                 W_bin = '{0:04b}'.format(round(W[7-j,i].item())+16)
#             for k in range(bit_precision):
#                 file.write(W_bin[k])
#             #file.write(' ')  # for visibility with blank between words, you can use
#         file.write('\n')
#     file.close() #close file
```

```
Start coding or generate with AI.
```

```
# ### Store psum ###
# ic_tile_id = 0
# oc_tile_id = 0


# kij = 0
# nij = 0

# # psum[len(ic_tileg), len(oc_tileg), array_size, len(p_nijg), len(kijg)]


# bit_precision = 16
# file = open('psum.txt', 'w') #write to file
# file.write('#time0col7[msb-lsb],time0col6[msb-lst],....,time0col0[msb-lst]#\n')
# file.write('#time1col7[msb-lsb],time1col6[msb-lst],....,time1col0[msb-lst]#\n')
# file.write('#................#\n')

# for kij in range(9):
#     psum_tile = psum[ic_tile_id,oc_tile_id,:,nij:nij+36,kij]
#     for i in range(psum_tile.size(1)):  # time step
#         for j in range(psum_tile.size(0)): # row #
#             if (psum_tile[7-j, i].item() >= 0):
#                 psum_tile_bin = '{0:016b}'.format(round(psum_tile[7-j,i].item()))
#             else:
#                 psum_tile_bin = '{0:016b}'.format(round(psum_tile[7-j,i].item())+65536)
#             #print(W_bin,' ', W[7-j,i].item(), j, i, '\n')
#             for k in range(bit_precision):
#                 file.write(psum_tile_bin[k])
#             #file.write(' ')  # for visibility with blank between words, you can use
#         file.write('\n')
# file.close() #close file
```

```
# psum_tile.shape
# print(psum_tile)
# print(out)
```

```
tensor([[   0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
           0.0000,   98.0000,  154.0000,  168.0000,   84.0000,    0.0000,
           0.0000,   77.0000,  147.0000,   91.0000,   49.0000,    0.0000,
           0.0000,   98.0000,  147.0000,  -21.0000,  -70.0000,    0.0000,
```

```
            0.0000,  161.0000,  147.0000,  -14.0000,   -7.0000,    0.0000,
            0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000],
        [   0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
            0.0000, -106.0000, -191.0000, -143.0000, -108.0000,    0.0000,
            0.0000, -101.0000, -123.0000,  -89.0000,  -49.0000,    0.0000,
            0.0000,  -50.0000,  -75.0000,  -25.0000,  -50.0000,    0.0000,
            0.0000,  -84.0000,  -85.0000,  -75.0000,  -67.0000,    0.0000,
            0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000],
        [   0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
            0.0000, -136.0000, -230.0000, -160.0000, -126.0000,    0.0000,
            0.0000, -133.0000, -147.0000,  -91.0000,  -49.0000,    0.0000,
            0.0000,  -64.0000,  -99.0000,  -35.0000,  -70.0000,    0.0000,
            0.0000, -103.0000, -111.0000, -104.0000,  -91.0000,    0.0000,
            0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000],
        [   0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
            0.0000,  112.0000,  224.0000,  154.0000,  126.0000,    0.0000,
            0.0000,  133.0000,  147.0000,   91.0000,   49.0000,    0.0000,
            0.0000,   28.0000,   63.0000,   35.0000,   70.0000,    0.0000,
            0.0000,   49.0000,   63.0000,   98.0000,   91.0000,    0.0000,
            0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000],
        [   0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
            0.0000, -126.0000,  -56.0000,   28.0000,    0.0000,    0.0000,
            0.0000,  -91.0000,  -21.0000,   77.0000,   49.0000,    0.0000,
            0.0000, -112.0000, -147.0000,  -35.0000,  -70.0000,    0.0000,
            0.0000, -147.0000, -175.0000, -112.0000,  -77.0000,    0.0000,
            0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000],
        [   0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
            0.0000, -126.0000, -238.0000, -168.0000, -126.0000,    0.0000,
            0.0000, -133.0000, -147.0000,  -91.0000,  -49.0000,    0.0000,
            0.0000, -112.0000, -147.0000,  -35.0000,  -70.0000,    0.0000,
            0.0000, -161.0000, -175.0000, -112.0000,  -91.0000,    0.0000,
            0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000],
        [   0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
            0.0000,  126.0000,  238.0000,  168.0000,  126.0000,    0.0000,
            0.0000,  133.0000,  147.0000,   91.0000,   49.0000,    0.0000,
            0.0000,  112.0000,  147.0000,   35.0000,   70.0000,    0.0000,
            0.0000,  161.0000,  175.0000,  112.0000,   91.0000,    0.0000,
            0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000],
        [   0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
            0.0000,  -75.0000, -134.0000,  -95.0000,  -69.0000,    0.0000,
            0.0000,  -89.0000,  -99.0000,  -42.0000,  -21.0000,    0.0000,
            0.0000,  -50.0000,  -84.0000,  -14.0000,  -20.0000,    0.0000,
            0.0000,  -73.0000,  -86.0000,  -53.0000,  -47.0000,    0.0000,
            0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000]],
       device='cuda:0', grad_fn=<SelectBackward0>)
tensor([[  392.0000,   161.0000,  -189.0000,  -336.0000,   468.9999,   357.0000,
             7.0000,   -63.0000,   510.9999,   231.0000,  -119.0000,   -28.0000,
           329.0000,    70.0000,  -294.0000,   -42.0000],
        [ -579.0000,  -907.0000,  -777.0000,  -428.0000,  -777.0000, -1188.0000,
         -1036.0000,  -539.0000,  -718.0000, -1027.0000,  -819.9999,  -448.0000,
          -431.0000,  -620.0000,  -535.0000,  -303.0000],
        [ -681.9999,  -941.0000,  -819.0000,  -434.0000,  -878.9999, -1187.0000,
         -1058.0000,  -538.0000,  -773.9999, -1075.9999,  -897.9999,  -446.0000,
          -576.9999,  -745.9999,  -624.0000,  -308.0000],
```

```
 1 # bit_precision = 16
 2 # file = open('out.txt', 'w') #write to file
 3 # file.write('#time0col7[msb-lsb],time0col6[msb-lst],....,time0col0[msb-lst]#\n')
 4 # file.write('#time1col7[msb-lsb],time1col6[msb-lst],....,time1col0[msb-lst]#\n')
 5 # file.write('#.............#\n')
 6
 7 # out_relu = relu(out)
 8 # print(out_relu)
 9 # for i in range(out_relu.size(1)):  # time step
10 #     for j in range(out_relu.size(0)): # row #
11 #         #Out_bin = '{0:016b}'.format(round(out[7-j,i].item()))
12 #         if (out[7-j, i].item() >= 0):
13 #             Out_bin = '{0:016b}'.format(round(out_relu[7-j,i].item()))
14 #         else:
15 #             Out_bin = '{0:016b}'.format(round(out_relu[7-j,i].item())+65536)
16 #         for k in range(bit_precision):
17 #             #print(X_bin[k])
18 #             file.write(Out_bin[k])
19 #        #file.write(' ')  # for visibility with blank between words, you can use
20 #     file.write('\n')
21 # file.close() #close file
```

```
tensor([[392.0000, 161.0000,   0.0000,   0.0000, 468.9999, 357.0000,   7.0000,
           0.0000, 510.9999, 231.0000,   0.0000,   0.0000, 329.0000,  70.0000,
           0.0000,   0.0000],
        [  0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,
           0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,
           0.0000,   0.0000],
        [  0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,
           0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,
           0.0000,   0.0000],
        [518.0000, 819.0000, 791.0000, 420.0000, 538.9999, 720.9999, 496.9999,
           0.0000, 245.0000, 469.0000, 385.0000,  84.0000, 147.0000, 196.0000,
```

```
          490.0000, 294.0000],
         [280.0000, 371.0000,   7.0000,   0.0000,   0.0000, 273.0000, 175.0000,
           63.0000,   0.0000, 105.0000, 370.9999, 252.0000,   0.0000,   0.0000,
           84.0000, 154.0000],
         [  0.0000,   0.0000,  91.0000, 238.0000,   7.0000,   0.0000,   0.0000,
            0.0000, 245.0000,   0.0000,   0.0000,   0.0000, 231.0000, 252.0000,
            0.0000,   0.0000],
         [461.9999, 511.0000, 371.0000,  84.0000, 427.0000, 622.9999, 343.0000,
            0.0000, 454.9999, 538.9999, 245.0000,   0.0000, 385.0000, 475.9999,
          224.0000,  70.0000],
         [  0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,
            0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,
            0.0000,   0.0000]], device='cuda:0', grad_fn=<ReluBackward0>)
```

```
# ##### The following code is just for verification #####
# ##### Please Ignore #####
# file = open('out.txt', 'r')
# result = []

# # Obtain the value in the last column
# for line in file:
#     line = line.strip()
#     if not line or line.startswith("#"):
#         continue
#     parts = line.split()
#     last_col = parts[-1]
#     result.append(last_col)

# print('The last column in out.txt is\n')
# print(result)

# def twos_complement_to_int(num_str, bits=16):
#     num_str = num_str.zfill(bits)
#     num = int(num_str, 2)
#     if num >= 2**(bits-1):
#         num -= 2**bits
#     return num

# print('\nConvert binary number into decimal number\n')
# result_int = [twos_complement_to_int(x) for x in result]
# print(result_int)
```

```
The last column in out.txt is

['0000000110001000', '0000000010100001', '1000000000000000', '1000000000000000', '0000000111010101', '0000000101100101', '00

Convert binary number into decimal number

[392, 161, -32768, -32768, 469, 357, 7, -32768, 511, 231, -32768, -32768, 329, 70, -32768, -32768]
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```